

Q1

2025-04-07

Question 1

a)

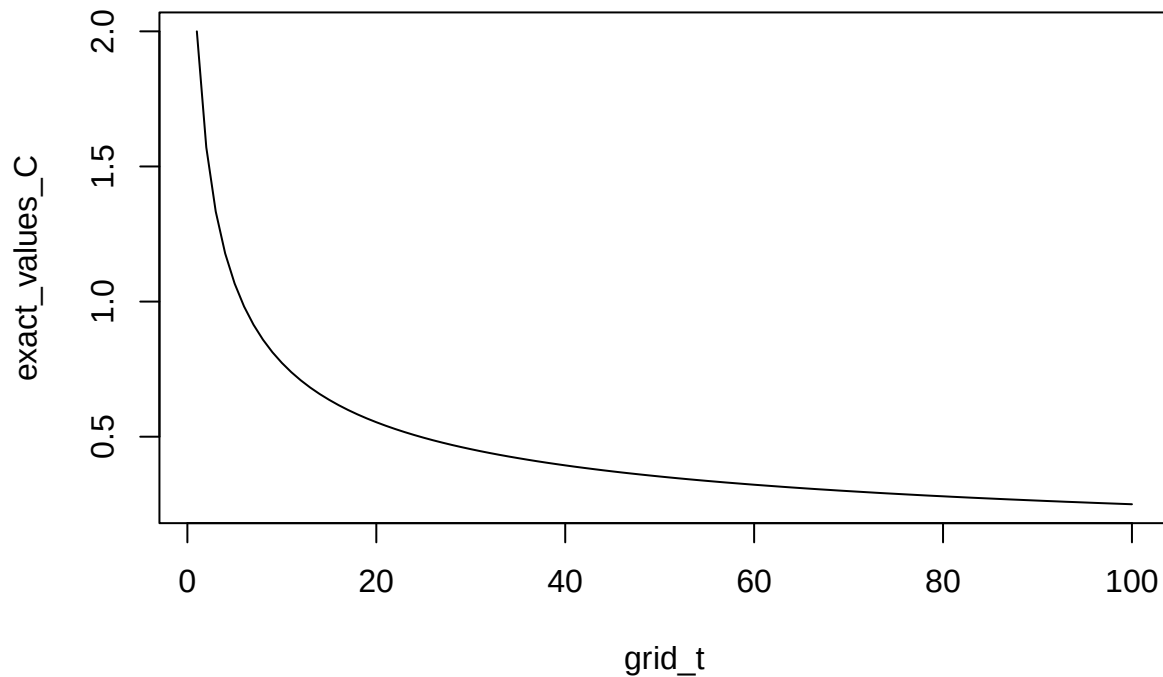
The true values

The following diagram shows the exact values of $C(t)$ for t from 1 to 100.

```
grid_t <- 1:100

exact_formula_C <- function(t){
  (sqrt(pi) * gamma((t + 1)/2))/gamma(t/2 + 1)
}

exact_values_C <- exact_formula_C(grid_t)
plot(grid_t, exact_values_C, type = 'l')
```



Monte Carlo Methods

```
n <- 10^6
```

Transformation 1 For transformation 1, we use $u = \sin x$, then $x = \arcsin(u)$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t dx \stackrel{x=\arcsin(u)}{=} \int_0^1 u^t [\cos(\arcsin(u))^{-1} + [\cos(\arcsin(u))]^{t-1}] du \stackrel{\mathcal{U} \stackrel{d}{=} \mathcal{U}^{(0,1)}}{=} \mathbb{E} \left[u^t [\cos(\arcsin(u))]^{-1} + [\cos(\arcsin(u))]^{t-1} \right]$$

```
set.seed(1)
EstIntegral_1 <- numeric(length(grid_t))
Diff_1 <- numeric(length(grid_t))
Std_Error_1 <- numeric(length(grid_t))

for (t in grid_t){
  u <- runif(n)
  sample_1 <- u^t*1/(cos(asin(u)))+(cos(asin(u)))^(t-1)
```

```

EstIntegral_1[t] <- mean(sample_1)
Diff_1[t] <- exact_formula_C(t) - EstIntegral_1[t]
Std_Error_1[t] <- sd(sample_1)/sqrt(n)
}

rbind(mean_1=mean(sample_1),mean_diff_1 = mean(Diff_1), mean_SE_1 = mean(Std_Error_1))

```

Using for loop to investigate the performance of integral across different values of t:

```

##                                [,1]
## mean_1                        0.2511193309
## mean_diff_1 -0.0003703208
## mean_SE_1      0.0026780224

```

Transformation 2 For transformation 2, we use $u = \cos x$, then $x = \arccos(u)$. When $x = 0$, then $u = 1$, and when $x = \pi/2$, then $u = 0$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t dx \stackrel{x=\arccos(u)}{=} \int_1^0 u^t [\sin(\arccos(u))]^{-1} + [\cos(\arccos(u))]^{t-1} du \stackrel{\mathcal{U} \stackrel{d}{=} \mathcal{U}(0,1)}{=} \mathbb{E} \left[u^t [\sin(\arccos(u))]^{-1} + [\cos(\arccos(u))]^{t-1} \right]$$

Using for loop to investigate the performance of integral across different values of t:

```

##                                [,1]
## mean_2                        0.2511193309
## mean_diff_2 -0.0003703208
## mean_SE_2      0.0026780224

```

According to the above results, transformation 1 and 2 have exactly the same mean and standard error.

Transformation 3 For transformation 3, we use $u = \frac{2}{\pi} * x$, then $x = \frac{\pi}{2}u$. When $x = 0$, then $u = 0$, and when $x = \pi/2$, then $u = 1$. We compute:

$$\int_0^{\pi/2} (\sin(x))^t + (\cos(x))^t dx \stackrel{x=\frac{\pi}{2}u}{=} \int_0^1 [(\sin(\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t)] \frac{\pi}{2} du \stackrel{\mathcal{U} \stackrel{d}{=} \mathcal{U}(0,1)}{=} \mathbb{E} \left[\frac{\pi}{2} [(\sin(\frac{\pi}{2}u))^t + ((\cos(\frac{\pi}{2}u))^t)] \right] \stackrel{M}{\approx}$$

Using for loop to investigate the performance of integral across different values of t:

```

set.seed(1)
EstIntegral_3 <- numeric(length(grid_t))
Diff_3 <- numeric(length(grid_t))
Std_Error_3 <- numeric(length(grid_t))

for (t in grid_t){
  u <- runif(n)

```

```

sample_3 <- ((sin(pi/2*u))^t + (cos(pi/2*u))^t) * pi/2

EstIntegral_3[t] <- mean(sample_3)
Diff_3[t] <- exact_formula_C(t) - EstIntegral_3[t]
Std_Error_3[t] <- sd(sample_3)/sqrt(n)
}

rbind(mean_diff_3 = mean(Diff_3), SE_3 = mean(Std_Error_3))

```

```

##           [,1]
## mean_diff_3 -1.242511e-05
## SE_3        4.949451e-04

```

```
EstIntegral_3
```

```

## [1] 2.0000681 1.5707963 1.3334925 1.1781235 1.0666586 0.9816143 0.9146422
## [8] 0.8588999 0.8130444 0.7731255 0.7389593 0.7091560 0.6818418 0.6580734
## [15] 0.6362084 0.6168822 0.5994613 0.5822990 0.5677819 0.5534041 0.5406744
## [22] 0.5284385 0.5173052 0.5062262 0.4976845 0.4859445 0.4778641 0.4694056
## [29] 0.4619554 0.4541905 0.4468209 0.4393570 0.4337940 0.4263221 0.4196799
## [36] 0.4157443 0.4093873 0.4035313 0.3994309 0.3941555 0.3891107 0.3840857
## [43] 0.3804467 0.3759447 0.3719446 0.3682730 0.3636654 0.3604153 0.3559723
## [50] 0.3524974 0.3499340 0.3455623 0.3421616 0.3395159 0.3357645 0.3339417
## [57] 0.3303444 0.3278584 0.3252950 0.3225415 0.3196360 0.3178809 0.3154077
## [64] 0.3128450 0.3094934 0.3076398 0.3050020 0.3021825 0.3012086 0.2986141
## [71] 0.2960704 0.2951546 0.2918595 0.2902953 0.2884126 0.2861079 0.2849094
## [78] 0.2833536 0.2813224 0.2786081 0.2776506 0.2752794 0.2751162 0.2720192
## [85] 0.2711570 0.2688321 0.2674860 0.2656036 0.2649343 0.2636464 0.2614405
## [92] 0.2596260 0.2595398 0.2584869 0.2567619 0.2552192 0.2541811 0.2513340
## [99] 0.2515884 0.2487829

```

The following table shows the comparison between different methods:

```

df_ts<-data.frame(
  Method=c("Transformation 1","Transformation 2","Transformation 3"),
  Mean=c(mean(sample_1),mean(sample_2),mean(sample_3)),
  Mean_diff=c(mean(Diff_1),mean(Diff_2),mean(Diff_3)),
  Standar_error=c(mean(Std_Error_1),mean(Std_Error_2),mean(Std_Error_3))
)
df_ts

```

```

##           Method      Mean      Mean_diff Standar_error
## 1 Transformation 1 0.2511193 -3.703208e-04 0.0026780224
## 2 Transformation 2 0.2511193 -3.703208e-04 0.0026780224
## 3 Transformation 3 0.2487829 -1.242511e-05 0.0004949451

```

According to the transformation methods above, transformation 3 has the smallest difference between estimated mean and actual mean, and also the smallest standard error. Thus we do the variance reduction based on transformation 3.

The antithetic variate reduction is not applicable since the function is not monotone. Then we try control variates method first.

1.Control variates method We have to estimate

$$\theta = \mathbb{E}(X) = \int_0^1 \frac{\pi}{2} [(\sin(\frac{\pi}{2}u))^t + (\cos(\frac{\pi}{2}u))^t] du, \text{ where } X = g(U) = \frac{\pi}{2} [(\sin(\frac{\pi}{2}u))^t + (\cos(\frac{\pi}{2}u))^t], \quad U \stackrel{d}{\sim} \mathcal{U}(0,1).$$

We want to find a random variable Y with mean 0 and a good constant c such that $\mathbb{E}(X + cY) \approx \theta$.

We consider the Taylor polynomial of g of degree 2 at 0:

$$T_1(u) = g(0) + g'(0)(u-0) + \frac{1}{2}g''(0)(u-0)^2 + O(u^3) = \frac{\pi}{2} - \frac{\pi^3 t}{16} u^2, \quad \text{where } g'(u) = \frac{\pi^2}{4} [t(\sin(\frac{\pi}{2}u))^{t-1} \cos(\frac{\pi}{2}u) - t(\cos(\frac{\pi}{2}u))^{t-1} \sin(\frac{\pi}{2}u)]$$

We take $T_1 = \frac{\pi}{2} - \frac{\pi^3 t}{16} u^2$. At this point, we define Y in terms of T_1 , but we need to have mean 0. Therefore, we compute:

$$\mathbb{E}(T_1) = \frac{\pi}{2} - \frac{\pi^3 t}{16} \mathbb{E}(u^2) = \frac{\pi}{2} - \frac{\pi^3 t}{16} * \frac{1}{3} = \frac{\pi}{2} - \frac{\pi^3}{48} t,$$

then we choose $Y = T_1 - (\frac{\pi}{2} - \frac{\pi^3 t}{48} t) = \frac{\pi}{2} - \frac{\pi^3 t}{16} u^2 - \frac{\pi}{2} + \frac{\pi^3 t}{48} t = \frac{\pi^3 t}{48} t - (\frac{\pi^3 t}{16} t) u^2$. We estimate θ via the following algorithm:

- Simulate $U_1, U_2, \dots, U_n \stackrel{d}{\sim} \mathcal{U}(0,1)$.
- Simulate $X: g(u_1), g(u_2), \dots, g(u_n)$.
- Simulate $Y: \frac{\pi^3 t}{48} t - (\frac{\pi^3 t}{16} t) u_1^2, \frac{\pi^3 t}{48} t - (\frac{\pi^3 t}{16} t) u_2^2, \dots, \frac{\pi^3 t}{48} t - (\frac{\pi^3 t}{16} t) u_n^2$.
- Calculate the $\text{Cov}(X, Y)$ and $\text{Var}(Y)$ with the samples to determine c^* .
- Approximate θ with the mean of $X + c^*Y$ and standard errors with the standard errors of $X + c^*Y$.

```
set.seed(1)
EstIntegral_cv1 <- numeric(length(grid_t))
Diff_cv1 <- numeric(length(grid_t))
SE_cv1 <- numeric(length(grid_t))

for (t in grid_t){
  u <- runif(n)
  X <- ((sin(pi/2*u))^t + (cos(pi/2*u))^t) * pi/2
  Y <- pi^3/48*t-pi^3/16*t*u^2
```

```

c <- -cov(X,Y)/var(Y)
X_cv1 <- X+c*Y
EstIntegral_cv1[t] <- mean(X_cv1)
Diff_cv1[t] <- EstIntegral_3[t] - mean(X_cv1)
SE_cv1[t] <- sd(X_cv1)/sqrt(n)
}

#cbind(Mean_diff_cv=Diff_cv1, SE_cv = mean(SE_cv1))
Diff_cv1

```

Using for loop to investigate the performance of integral across different values of t:

```

## [1] 1.699058e-05 0.000000e+00 -2.669888e-06 -1.993033e-05 8.080811e-06
## [6] 3.767312e-05 1.084084e-04 -1.268224e-04 1.699582e-04 -3.072084e-05
## [11] 1.193494e-04 -1.496409e-05 4.933125e-05 3.962410e-04 1.154123e-04
## [16] 4.194221e-05 -1.008114e-04 3.760437e-05 4.834334e-05 1.214842e-04
## [21] -1.642510e-05 1.395166e-04 -1.623889e-05 -7.848970e-05 7.231963e-07
## [26] -7.869360e-05 -6.249720e-05 1.118707e-04 -5.968964e-05 -3.293536e-04
## [31] -7.070315e-05 -4.926082e-05 -1.995650e-05 2.410614e-04 -4.061236e-05
## [36] 1.138309e-04 -3.335209e-04 -2.313230e-05 8.061175e-05 -4.539678e-05
## [41] 6.959658e-05 1.461323e-05 -2.437679e-04 -3.373867e-06 1.809683e-04
## [46] 1.809839e-04 2.195743e-05 -1.053763e-04 -1.310645e-04 -2.686809e-04
## [51] 5.746016e-05 -1.572428e-04 -1.890205e-05 -7.603832e-05 -3.195134e-04
## [56] 1.763864e-04 1.910307e-04 2.283433e-04 -4.027479e-05 1.236303e-04
## [61] -3.865897e-05 1.842171e-05 2.597220e-04 -8.834013e-05 -8.640819e-05
## [66] -5.864226e-05 1.735729e-04 5.588077e-06 1.729983e-04 -1.676521e-05
## [71] 1.957201e-04 6.162763e-05 9.527570e-05 5.088602e-05 1.360552e-05
## [76] 2.455240e-05 1.009907e-04 -1.171777e-05 4.443303e-05 -4.044264e-05
## [81] -3.571053e-05 -9.029018e-05 3.485581e-05 -4.297194e-05 -1.132978e-04
## [86] 9.581251e-06 -1.106921e-04 -9.609839e-05 1.553873e-04 -3.900028e-05
## [91] 7.712500e-05 1.934715e-04 -1.635527e-04 -1.045734e-04 -1.208624e-04
## [96] 4.254007e-06 -4.242943e-05 -1.332535e-04 3.261789e-05 -3.508264e-05

```

```
print('-----')
```

```
## [1] "-----"
```

```
SE_cv1
```

```

## [1] 1.892447e-04 1.380675e-19 1.570716e-04 2.694888e-04 3.487444e-04
## [6] 4.042856e-04 4.435561e-04 4.720219e-04 4.919495e-04 5.065608e-04
## [11] 5.173524e-04 5.250467e-04 5.304966e-04 5.344073e-04 5.368642e-04
## [16] 5.391474e-04 5.407164e-04 5.412861e-04 5.415383e-04 5.412063e-04
## [21] 5.407446e-04 5.398704e-04 5.397368e-04 5.387782e-04 5.378931e-04
## [26] 5.363922e-04 5.352046e-04 5.344912e-04 5.326623e-04 5.315563e-04

```

```
## [31] 5.295807e-04 5.279160e-04 5.274216e-04 5.256965e-04 5.234954e-04
## [36] 5.228676e-04 5.219040e-04 5.194771e-04 5.186005e-04 5.173186e-04
## [41] 5.154044e-04 5.141655e-04 5.132514e-04 5.111880e-04 5.104460e-04
## [46] 5.090477e-04 5.072990e-04 5.063203e-04 5.046353e-04 5.034555e-04
## [51] 5.025375e-04 5.004622e-04 4.990238e-04 4.982829e-04 4.965251e-04
## [56] 4.964605e-04 4.941452e-04 4.934973e-04 4.927475e-04 4.911011e-04
## [61] 4.896659e-04 4.894634e-04 4.882578e-04 4.870653e-04 4.850323e-04
## [66] 4.849632e-04 4.832340e-04 4.815683e-04 4.816205e-04 4.801368e-04
## [71] 4.787301e-04 4.787186e-04 4.766365e-04 4.758539e-04 4.750061e-04
## [76] 4.738417e-04 4.732335e-04 4.725728e-04 4.714464e-04 4.694532e-04
## [81] 4.691003e-04 4.679570e-04 4.681920e-04 4.661160e-04 4.660904e-04
## [86] 4.644448e-04 4.636615e-04 4.624801e-04 4.621531e-04 4.613092e-04
## [91] 4.598961e-04 4.581956e-04 4.589605e-04 4.583411e-04 4.574421e-04
## [96] 4.562704e-04 4.556844e-04 4.538144e-04 4.544333e-04 4.522637e-04
```

```
print('-----')
```

```
## [1] "-----"
```

```
Std_Error_3
```

```
## [1] 1.953755e-04 1.380675e-19 1.618624e-04 2.775972e-04 3.592291e-04
## [6] 4.165828e-04 4.571114e-04 4.863212e-04 5.074983e-04 5.224552e-04
## [11] 5.337684e-04 5.417982e-04 5.475632e-04 5.519143e-04 5.542845e-04
## [16] 5.566346e-04 5.579778e-04 5.587070e-04 5.590500e-04 5.589006e-04
## [21] 5.580307e-04 5.576045e-04 5.568826e-04 5.560161e-04 5.549048e-04
## [26] 5.535146e-04 5.519663e-04 5.513336e-04 5.494950e-04 5.479214e-04
## [31] 5.462729e-04 5.445349e-04 5.437659e-04 5.418837e-04 5.397819e-04
## [36] 5.393421e-04 5.375099e-04 5.355645e-04 5.346181e-04 5.327322e-04
## [41] 5.311910e-04 5.295928e-04 5.282930e-04 5.264842e-04 5.256692e-04
## [46] 5.243126e-04 5.220625e-04 5.209660e-04 5.194352e-04 5.177958e-04
## [51] 5.170690e-04 5.149151e-04 5.133253e-04 5.125354e-04 5.105864e-04
## [56] 5.104600e-04 5.084026e-04 5.076857e-04 5.065325e-04 5.049305e-04
## [61] 5.034696e-04 5.029862e-04 5.020885e-04 5.003983e-04 4.984453e-04
## [66] 4.980320e-04 4.965418e-04 4.946301e-04 4.949019e-04 4.929174e-04
## [71] 4.917388e-04 4.915480e-04 4.893814e-04 4.887331e-04 4.875099e-04
## [76] 4.862632e-04 4.854236e-04 4.849297e-04 4.838103e-04 4.819424e-04
## [81] 4.812574e-04 4.800345e-04 4.802369e-04 4.780260e-04 4.779304e-04
## [86] 4.763261e-04 4.752997e-04 4.742280e-04 4.737600e-04 4.728135e-04
## [91] 4.714172e-04 4.698949e-04 4.703429e-04 4.697348e-04 4.688431e-04
## [96] 4.676307e-04 4.668371e-04 4.647661e-04 4.657543e-04 4.631303e-04
```

```
mean(Std_Error_3)
```

```
## [1] 0.0004949451
```

```

set.seed(1)
EstIntegral_cv2 <- numeric(length(grid_t))
Diff_cv2 <- numeric(length(grid_t))
SE_cv2 <- numeric(length(grid_t))

for (t in grid_t){
  u <- runif(n)
  X_tilde <- ((sin(pi/2*u + pi/4))^t) * pi/2
  Y_2 <- X_tilde - mean(X_tilde)
  c <- -cov(X,Y_2)/var(Y_2)

  theta_hat <- X+c*Y_2
  EstIntegral_cv2[t] <- mean(theta_hat)
  SE_cv2[t] <- sd(theta_hat)/sqrt(n)
  Diff_cv2[t] <- EstIntegral_3[t] - mean(theta_hat)
}

# cbind(mean = mean_cv2, Mean_diff_cv=Diff_cv2, SE_cv = mean(SE_cv2))
# improve_1 <- abs(mean(SE_cv2)-mean(Std_Error_3))/mean(Std_Error_3)
# improve_1
# cat("The control variate method reduced the standard error by",improve_1)

Diff_cv2

```

```

## [1] 1.751285e+00 1.322013e+00 1.084710e+00 9.293406e-01 8.178757e-01
## [6] 7.328313e-01 6.658593e-01 6.101170e-01 5.642615e-01 5.243426e-01
## [11] 4.901764e-01 4.603731e-01 4.330589e-01 4.092905e-01 3.874255e-01
## [16] 3.680993e-01 3.506784e-01 3.335161e-01 3.189990e-01 3.046212e-01
## [21] 2.918915e-01 2.796556e-01 2.685223e-01 2.574432e-01 2.489015e-01
## [26] 2.371616e-01 2.290811e-01 2.206227e-01 2.131725e-01 2.054076e-01
## [31] 1.980380e-01 1.905740e-01 1.850111e-01 1.775392e-01 1.708970e-01
## [36] 1.669614e-01 1.606044e-01 1.547484e-01 1.506480e-01 1.453726e-01
## [41] 1.403278e-01 1.353028e-01 1.316638e-01 1.271618e-01 1.231617e-01
## [46] 1.194901e-01 1.148824e-01 1.116324e-01 1.071894e-01 1.037145e-01
## [51] 1.011511e-01 9.677936e-02 9.337867e-02 9.073295e-02 8.698155e-02
## [56] 8.515882e-02 8.156150e-02 7.907550e-02 7.651209e-02 7.375858e-02
## [61] 7.085311e-02 6.909796e-02 6.662477e-02 6.406206e-02 6.071047e-02
## [66] 5.885689e-02 5.621914e-02 5.339962e-02 5.242567e-02 4.983119e-02
## [71] 4.728747e-02 4.637168e-02 4.307659e-02 4.151238e-02 3.962965e-02
## [76] 3.732496e-02 3.612648e-02 3.457071e-02 3.253952e-02 2.982517e-02
## [81] 2.886769e-02 2.649654e-02 2.633328e-02 2.323633e-02 2.237413e-02
## [86] 2.004920e-02 1.870309e-02 1.682069e-02 1.615143e-02 1.486345e-02
## [91] 1.265757e-02 1.084311e-02 1.075691e-02 9.703962e-03 7.978981e-03
## [96] 6.436254e-03 5.398232e-03 2.551085e-03 2.805497e-03 5.551115e-17

```

```
print('-----')
```

```
## [1] "-----"
```

```
SE_cv2
```

```
## [1] 0.0004631300 0.0004631299 0.0004631299 0.0004631303 0.0004631303
## [6] 0.0004631302 0.0004631302 0.0004631302 0.0004631295 0.0004631302
## [11] 0.0004631303 0.0004631303 0.0004631303 0.0004631302 0.0004631302
## [16] 0.0004631303 0.0004631303 0.0004631303 0.0004631302 0.0004631303
## [21] 0.0004631303 0.0004631303 0.0004631303 0.0004631302 0.0004631302
## [26] 0.0004631302 0.0004631294 0.0004631302 0.0004631301 0.0004631302
## [31] 0.0004631302 0.0004631303 0.0004631303 0.0004631300 0.0004631300
## [36] 0.0004631303 0.0004631302 0.0004631303 0.0004631303 0.0004631303
## [41] 0.0004631300 0.0004631302 0.0004631302 0.0004631296 0.0004631300
## [46] 0.0004631303 0.0004631301 0.0004631303 0.0004631303 0.0004631299
## [51] 0.0004631303 0.0004631300 0.0004631300 0.0004631302 0.0004631302
## [56] 0.0004631297 0.0004631300 0.0004631303 0.0004631298 0.0004631290
## [61] 0.0004631303 0.0004631288 0.0004631299 0.0004631301 0.0004631301
## [66] 0.0004631302 0.0004631303 0.0004631303 0.0004631302 0.0004631290
## [71] 0.0004631303 0.0004631303 0.0004631302 0.0004631302 0.0004631303
## [76] 0.0004631303 0.0004631303 0.0004631303 0.0004631303 0.0004631300
## [81] 0.0004631301 0.0004631303 0.0004631303 0.0004631301 0.0004631303
## [86] 0.0004631303 0.0004631302 0.0004631303 0.0004631303 0.0004631303
## [91] 0.0004631302 0.0004631303 0.0004631302 0.0004631292 0.0004631297
## [96] 0.0004631303 0.0004631300 0.0004631303 0.0004631303 0.0004433078
```

```
print('-----')
```

```
## [1] "-----"
```

```
Std_Error_3
```

```
## [1] 1.953755e-04 1.380675e-19 1.618624e-04 2.775972e-04 3.592291e-04
## [6] 4.165828e-04 4.571114e-04 4.863212e-04 5.074983e-04 5.224552e-04
## [11] 5.337684e-04 5.417982e-04 5.475632e-04 5.519143e-04 5.542845e-04
## [16] 5.566346e-04 5.579778e-04 5.587070e-04 5.590500e-04 5.589006e-04
## [21] 5.580307e-04 5.576045e-04 5.568826e-04 5.560161e-04 5.549048e-04
## [26] 5.535146e-04 5.519663e-04 5.513336e-04 5.494950e-04 5.479214e-04
## [31] 5.462729e-04 5.445349e-04 5.437659e-04 5.418837e-04 5.397819e-04
## [36] 5.393421e-04 5.375099e-04 5.355645e-04 5.346181e-04 5.327322e-04
## [41] 5.311910e-04 5.295928e-04 5.282930e-04 5.264842e-04 5.256692e-04
## [46] 5.243126e-04 5.220625e-04 5.209660e-04 5.194352e-04 5.177958e-04
## [51] 5.170690e-04 5.149151e-04 5.133253e-04 5.125354e-04 5.105864e-04
```

```
## [56] 5.104600e-04 5.084026e-04 5.076857e-04 5.065325e-04 5.049305e-04
## [61] 5.034696e-04 5.029862e-04 5.020885e-04 5.003983e-04 4.984453e-04
## [66] 4.980320e-04 4.965418e-04 4.946301e-04 4.949019e-04 4.929174e-04
## [71] 4.917388e-04 4.915480e-04 4.893814e-04 4.887331e-04 4.875099e-04
## [76] 4.862632e-04 4.854236e-04 4.849297e-04 4.838103e-04 4.819424e-04
## [81] 4.812574e-04 4.800345e-04 4.802369e-04 4.780260e-04 4.779304e-04
## [86] 4.763261e-04 4.752997e-04 4.742280e-04 4.737600e-04 4.728135e-04
## [91] 4.714172e-04 4.698949e-04 4.703429e-04 4.697348e-04 4.688431e-04
## [96] 4.676307e-04 4.668371e-04 4.647661e-04 4.657543e-04 4.631303e-04
```

```
mean(SE_cv2)
```

```
## [1] 0.0004629319
```

```
set.seed(1)
n <- 10^6
u <- runif(n)
j <- 1:n
sample_1a2 <- ((sin(pi/2*((u+j-1)/n)))^t + (cos(pi/2*((u+j-1)/n)))^t) * pi/2

Mu_ss1 <- mean(sample_1a2)
Diff_ss1 <- EstIntegral_3 - Mu_ss1
SE_ss1 <- sd(sample_1a2)/sqrt(n)

cbind(Mu_ss1, Diff_ss1, SE_ss1)
```

2.Stratified Sampling

```
##           Mu_ss1      Diff_ss1      SE_ss1
## [1,] 0.250037  1.750031154 0.000464274
## [2,] 0.250037  1.320759363 0.000464274
## [3,] 0.250037  1.083455538 0.000464274
## [4,] 0.250037  0.928086568 0.000464274
## [5,] 0.250037  0.816621614 0.000464274
## [6,] 0.250037  0.731577292 0.000464274
## [7,] 0.250037  0.664605197 0.000464274
## [8,] 0.250037  0.608862953 0.000464274
## [9,] 0.250037  0.563007480 0.000464274
## [10,] 0.250037 0.523088564 0.000464274
## [11,] 0.250037 0.488922333 0.000464274
## [12,] 0.250037 0.459119012 0.000464274
## [13,] 0.250037 0.431804829 0.000464274
## [14,] 0.250037 0.408036450 0.000464274
## [15,] 0.250037 0.386171412 0.000464274
```

##	[16,]	0.250037	0.366845264	0.000464274
##	[17,]	0.250037	0.349424319	0.000464274
##	[18,]	0.250037	0.332262042	0.000464274
##	[19,]	0.250037	0.317744979	0.000464274
##	[20,]	0.250037	0.303367144	0.000464274
##	[21,]	0.250037	0.290637460	0.000464274
##	[22,]	0.250037	0.278401498	0.000464274
##	[23,]	0.250037	0.267268232	0.000464274
##	[24,]	0.250037	0.256189193	0.000464274
##	[25,]	0.250037	0.247647491	0.000464274
##	[26,]	0.250037	0.235907530	0.000464274
##	[27,]	0.250037	0.227827091	0.000464274
##	[28,]	0.250037	0.219368641	0.000464274
##	[29,]	0.250037	0.211918451	0.000464274
##	[30,]	0.250037	0.204153512	0.000464274
##	[31,]	0.250037	0.196783931	0.000464274
##	[32,]	0.250037	0.189319994	0.000464274
##	[33,]	0.250037	0.183757060	0.000464274
##	[34,]	0.250037	0.176285169	0.000464274
##	[35,]	0.250037	0.169642921	0.000464274
##	[36,]	0.250037	0.165707361	0.000464274
##	[37,]	0.250037	0.159350346	0.000464274
##	[38,]	0.250037	0.153494328	0.000464274
##	[39,]	0.250037	0.149393969	0.000464274
##	[40,]	0.250037	0.144118579	0.000464274
##	[41,]	0.250037	0.139073757	0.000464274
##	[42,]	0.250037	0.134048779	0.000464274
##	[43,]	0.250037	0.130409726	0.000464274
##	[44,]	0.250037	0.125907748	0.000464274
##	[45,]	0.250037	0.121907605	0.000464274
##	[46,]	0.250037	0.118236020	0.000464274
##	[47,]	0.250037	0.113628388	0.000464274
##	[48,]	0.250037	0.110378301	0.000464274
##	[49,]	0.250037	0.105935331	0.000464274
##	[50,]	0.250037	0.102460464	0.000464274
##	[51,]	0.250037	0.099897018	0.000464274
##	[52,]	0.250037	0.095525305	0.000464274
##	[53,]	0.250037	0.092124615	0.000464274
##	[54,]	0.250037	0.089478890	0.000464274
##	[55,]	0.250037	0.085727497	0.000464274
##	[56,]	0.250037	0.083904764	0.000464274
##	[57,]	0.250037	0.080307441	0.000464274
##	[58,]	0.250037	0.077821445	0.000464274
##	[59,]	0.250037	0.075258036	0.000464274
##	[60,]	0.250037	0.072504527	0.000464274
##	[61,]	0.250037	0.069599053	0.000464274
##	[62,]	0.250037	0.067843902	0.000464274
##	[63,]	0.250037	0.065370719	0.000464274

```
## [64,] 0.250037 0.062808009 0.000464274
## [65,] 0.250037 0.059456411 0.000464274
## [66,] 0.250037 0.057602840 0.000464274
## [67,] 0.250037 0.054965081 0.000464274
## [68,] 0.250037 0.052145568 0.000464274
## [69,] 0.250037 0.051171616 0.000464274
## [70,] 0.250037 0.048577134 0.000464274
## [71,] 0.250037 0.046033411 0.000464274
## [72,] 0.250037 0.045117623 0.000464274
## [73,] 0.250037 0.041822531 0.000464274
## [74,] 0.250037 0.040258325 0.000464274
## [75,] 0.250037 0.038375591 0.000464274
## [76,] 0.250037 0.036070905 0.000464274
## [77,] 0.250037 0.034872427 0.000464274
## [78,] 0.250037 0.033316659 0.000464274
## [79,] 0.250037 0.031285467 0.000464274
## [80,] 0.250037 0.028571113 0.000464274
## [81,] 0.250037 0.027613635 0.000464274
## [82,] 0.250037 0.025242484 0.000464274
## [83,] 0.250037 0.025079229 0.000464274
## [84,] 0.250037 0.021982276 0.000464274
## [85,] 0.250037 0.021120078 0.000464274
## [86,] 0.250037 0.018795141 0.000464274
## [87,] 0.250037 0.017449032 0.000464274
## [88,] 0.250037 0.015566631 0.000464274
## [89,] 0.250037 0.014897372 0.000464274
## [90,] 0.250037 0.013609393 0.000464274
## [91,] 0.250037 0.011403517 0.000464274
## [92,] 0.250037 0.009589058 0.000464274
## [93,] 0.250037 0.009502851 0.000464274
## [94,] 0.250037 0.008449907 0.000464274
## [95,] 0.250037 0.006724926 0.000464274
## [96,] 0.250037 0.005182199 0.000464274
## [97,] 0.250037 0.004144177 0.000464274
## [98,] 0.250037 0.001297030 0.000464274
## [99,] 0.250037 0.001551442 0.000464274
## [100,] 0.250037 -0.001254055 0.000464274
```

```
improve_SS <- abs(Mu_ss1-EstIntegral_3)/EstIntegral_3
```

The following table shows the comparison between different variance reduction methods,

```
df_vr1<-data.frame(
  Method=c("Transformation 3","VC_1a","SS_1a"),
  Mean=c(mean(sample_3),mean(EstIntegral_2),mean(EstIntegral_3)),
  Mean_diff=c(mean(Diff_3),mean(Diff_cv2),mean(Diff_ss1)),
  Standar_error=c(mean(Std_Error_3),mean(SE_cv2),mean(SE_ss1))
```

```
)  
df_vr1
```

```
##           Method      Mean      Mean_diff Standar_error  
## 1 Transformation 3 0.2487829 -1.242511e-05  0.0004949451  
## 2              VC_1a 0.4527820  2.036412e-01  0.0004629319  
## 3              SS_1a 0.4524241  2.023871e-01  0.0004642740
```

(b)

Calculate the true value of the double integral

```
set.seed(1)  
f <- function(x, y) {  
  r <- sqrt(x^2 + y^2)  
  ifelse(r == 0, 1, sin(r) / r) # Handle the removable singularity at (0,0)  
}  
  
result <- integrate(function(y) {  
  sapply(y, function(yi) {  
    integrate(function(x) f(x, yi), -1, 1)$value  
  })  
}, -1, 1)  
  
# Print the result  
true_integral_1b <- result$value  
true_integral_1b
```

```
## [1] 3.575761
```

```
set.seed(1)  
# install.packages('pracma')  
library(pracma)  
  
f <- function(x, y) {  
  r <- sqrt(x^2 + y^2)  
  ifelse(r == 0, 1, sin(r) / r)  
}  
  
dblquad(f, -1, 1, -1, 1)
```

Alternatively, using another package:

```
## [1] 3.575761
```

Pure Monte Carlo Methods

Since the equation is symmetric, we can compute:

$$\int_{-1}^1 \int_{-1}^1 \frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} dx dy = 4 \int_0^1 \int_0^1 \frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} dx dy \stackrel{(X,Y) \text{ i.i.d. } \mathcal{U}(0,1)}{=} 4 \mathbb{E}\left[\frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}}\right] \stackrel{MC}{\approx} \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{x_i^2 + y_i^2})}{\sqrt{x_i^2 + y_i^2}}$$

where $X_1, \dots, X_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, $Y_1, \dots, Y_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, and the samples are independent.

```
set.seed(1)

n <- 10^5
x <- runif(n)
y <- runif(n)
sample_mc <- 4 * sin(sqrt(x^2 + y^2))/sqrt(x^2 + y^2)

EstIntegral_mc <- mean(sample_mc)
Diff_mc <- true_integral_1b - EstIntegral_mc
Std_Error_mc <- sd(sample_mc)/sqrt(n)

rbind(EstIntegral_mc, Diff_mc, Std_Error_mc)
```

```
##               [,1]
## EstIntegral_mc 3.575741e+00
## Diff_mc        2.059416e-05
## Std_Error_mc   8.211745e-04
```

1. Antithetic Variate Reduction We need to calculate

$$\frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} = \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{((1-x)^2 + (1-y)^2)})}{\sqrt{(1-x)^2 + (1-y)^2}},$$

where $X_1, \dots, X_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, $Y_1, \dots, Y_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, and the samples are independent.

```
set.seed(1)
n <- 10^5
x <- runif(n)
y <- runif(n)
sample1 <- 4 * sin(sqrt(x^2 + y^2))/sqrt(x^2 + y^2)
sample2 <- 4 * sin(sqrt((1-x)^2 + (1-y)^2))/sqrt((1-x)^2 + (1-y)^2)
EstIntegral_av <- mean((sample1+sample2)/2)
Mean_diff_av <- true_integral_1b - EstIntegral_av
Std_Error_av <- sd((sample1+sample2)/2)/sqrt(n)

rbind(EstIntegral_av, Mean_diff_av, Std_Error_av)
```

```
##                                [,1]
## EstIntegral_av 3.5755699677
## Mean_diff_av   0.0001912936
## Std_Error_av   0.0001980729
```

```
improve_av <- abs(Std_Error_av-Std_Error_mc)/Std_Error_mc
improve_av
```

```
## [1] 0.7587932
```

The antithetic variate reduction reduces the standard error by 75.8793%.

2.Stratified Sampling Since $1 - U_i \stackrel{d}{=} U_i$, we can compute

$$h(U) = \frac{4}{N} \sum_{i=1}^N \frac{\sin(\sqrt{((\frac{U_i+i-1}{n})^2 + ((\frac{U_j+j-1}{n})^2)})}{\sqrt{((\frac{U_i+i-1}{n})^2 + ((\frac{U_j+j-1}{n})^2)}},$$

where $X_1, \dots, X_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, $Y_1, \dots, Y_N \stackrel{d}{\sim} \mathcal{U}(0, 1)$, and the samples are independent.

```
set.seed(1)
n <- 10^5
u <- runif(n)
j <- 1:n
sample_1b2 <- 4 * (sin(sqrt(((u+j-1)/n)^2 + ((u+j-1)/n)^2)))/sqrt(((u+j-1)/n)^2 + ((u+j-1)/n)^2)

Mu_SS <- mean(sample_1b2)
Diff_SS <- true_integral_1b - Mu_SS
SE_SS <- sd(sample_1b2)/sqrt(n)

rbind(Mu_SS, Diff_SS, SE_SS)
```

```
##                                [,1]
## Mu_SS      3.581334509
## Diff_SS    -0.005573247
## SE_SS      0.001153904
```

```
improve_SS <- abs(Mu_SS-EstIntegral_mc)/EstIntegral_mc
improve_SS
```

```
## [1] 0.001564387
```

The following table shows the comparison between different methods:

```
df_1b<-data.frame(
  Method=c("Ture Value_1b","MC_1b","AV_1b","SS_1b"),
  Mean=c(true_integral_1b,mean(EstIntegral_mc),mean(EstIntegral_av),mean(Mu_SS)),
  Mean_diff=c("NA",mean(Diff_mc),mean(Mean_diff_av),mean(Diff_SS)),
  Standar_error=c("NA",mean(Std_Error_mc),mean(Std_Error_av),mean(SE_SS))
)
df_1b
```

##	Method	Mean	Mean_diff	Standar_error
## 1	Ture Value_1b	3.575761	NA	NA
## 2	MC_1b	3.575741	2.05941642512641e-05	0.000821174490111747
## 3	AV_1b	3.575570	0.000191293603578391	0.000198072897562495
## 4	SS_1b	3.581335	-0.00557324748805099	0.00115390410753427

Therefore, the antithetic variate reduction can improve the accuaracy by 75.8793%, which is the best.